

REGN NO: FA25-BCS-017

NAME:FAREEHA BATOOL

COURSE: DATA STRUCTURE LAB

LAB TASK

SINGLY LINKED LISTS

TASK 01:

```
#include <iostream>
```

```
using namespace std;
```

```
// Node structure
```

```
struct Node
```

```
{
```

```
    int data;
```

```
    Node* next;
```

```
};
```

```
// Insert node at end
```

```
void insertEnd(Node*& head, int value)
```

```
{
```

```
    Node* newNode = new Node;
```

```
    newNode->data = value;
```

```
    newNode->next = NULL;
```

```
    if (head == NULL)
```

```
    {
```

```
        head = newNode;
```

```
        return;
```

```
    }
```

```
    Node* temp = head;
```

```
while (temp->next != NULL)
{
    temp = temp->next;
}

temp->next = newNode;
}

// Display Linked List
void display(Node* head)
{
    Node* temp = head;

    while (temp != NULL)
    {
        cout << temp->data << " ";
        temp = temp->next;
    }

    cout << endl;
}

// Reverse using Loop
void reverseUsingLoop(Node* head)
{
    int arr[100];
    int count = 0;
```

```
Node* temp = head;

while (temp != NULL)
{
    arr[count] = temp->data;
    count++;

    temp = temp->next;
}

cout << "Reverse using Loop: ";

for (int i = count - 1; i >= 0; i--)
{
    cout << arr[i] << " ";
}

cout << endl;
}

// Reverse using Recursion
void reverseUsingRecursion(Node* head)
{
    if (head == NULL)
    {
        return;
    }

    reverseUsingRecursion(head->next);
```

```
        cout << head->data << " ";
    }
int main()
{
    Node* head = NULL;

    // Create Linked List
    insertEnd(head, 10);
    insertEnd(head, 20);
    insertEnd(head, 30);
    insertEnd(head, 40);
    insertEnd(head, 50);

    // Original List
    cout << "Original Linked List: ";
    display(head);

    // Reverse using Loop
    reverseUsingLoop(head);

    // Reverse using Recursion
    cout << "Reverse using Recursion: ";
    reverseUsingRecursion(head);
    cout << endl;

    return 0;
}
```

OUTPUT:

```
C:\Users\HP\OneDrive\Desktop >
Original Linked List: 10 20 30 40 50
Reverse using Loop: 50 40 30 20 10
Reverse using Recursion: 50 40 30 20 10

-----
Process exited after 0.5035 seconds with return value 0
Press any key to continue . . . |
```

TASK 02:

```
#include using namespace std;
```

```
// Node structure struct Node { int data; Node* next; };
```

```
// Insert node at end void insertEnd(Node*& head, int value) { Node* newNode = new Node;
```

```
newNode->data = value;
```

```
newNode->next = NULL;
```

```
if (head == NULL)
```

```
{
```

```
    head = newNode;
```

```
    return;
```

```
}
```

```
Node* temp = head;
```

```
while (temp->next != NULL)
```

```
{
```

```
    temp = temp->next;
```

```
}
```

```
temp->next = newNode;
```

```
}
```

```
// Display Linked List void display(Node* head) { Node* temp = head;
```

```
while (temp != NULL)
```

```
{
```

```

        cout << temp->data << " ";
        temp = temp->next;
    }

    cout << endl;

}

Node* mergeLists(Node* head1, Node* head2) { Node* head3 = NULL;
// Copy first list into third list
Node* temp = head1;

while (temp != NULL)
{
    insertEnd(head3, temp->data);
    temp = temp->next;
}

// Copy second list into third list
temp = head2;

while (temp != NULL)
{
    insertEnd(head3, temp->data);
    temp = temp->next;
}

return head3;

}

int main() { // First Linked List Node* list1 = NULL;
insertEnd(list1, 10);
insertEnd(list1, 20);
insertEnd(list1, 30);

// Second Linked List
Node* list2 = NULL;

```

```

insertEnd(list2, 40);
insertEnd(list2, 50);
insertEnd(list2, 60);

// Display first list
cout << "List 1: ";
display(list1);

// Display second list
cout << "List 2: ";
display(list2);

// Merge into third list
Node* list3 = mergeLists(list1, list2);

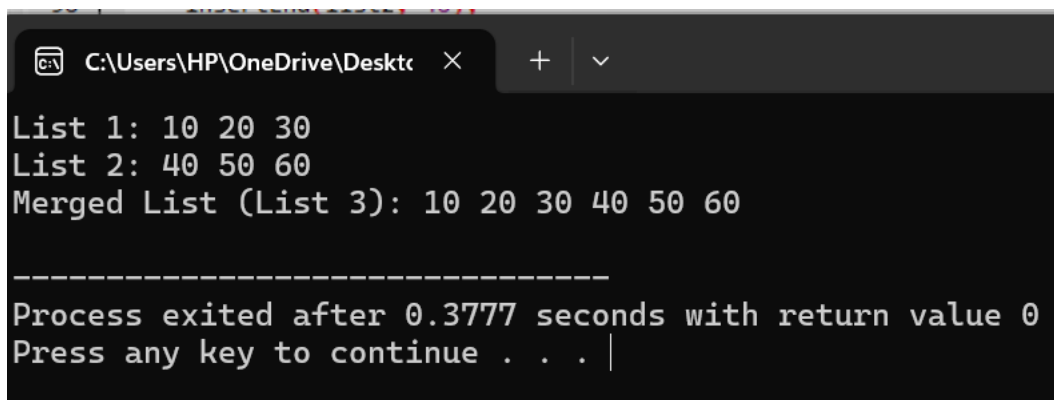
// Display third list
cout << "Merged List (List 3): ";
display(list3);

return 0;

}

```

OUTPUT:



```

C:\Users\HP\OneDrive\Desktop
List 1: 10 20 30
List 2: 40 50 60
Merged List (List 3): 10 20 30 40 50 60

-----
Process exited after 0.3777 seconds with return value 0
Press any key to continue . . . |

```

TASK 3:

```

#include <iostream>

using namespace std;

```

```
// Node structure
```

```
struct Node
```

```
{  
    int data;  
    Node* next;  
};
```

```
// Insert node at end
```

```
void insertEnd(Node*& head, int value)
```

```
{  
    Node* newNode = new Node;
```

```
    newNode->data = value;
```

```
    newNode->next = NULL;
```

```
    if (head == NULL)
```

```
    {  
        head = newNode;  
        return;  
    }
```

```
    Node* temp = head;
```

```
    while (temp->next != NULL)
```

```
    {  
        temp = temp->next;  
    }
```

```
temp->next = newNode;
}

// Display Linked List
void display(Node* head)
{
    Node* temp = head;

    while (temp != NULL)
    {
        cout << temp->data << " ";
        temp = temp->next;
    }

    cout << endl;
}

void findOccurrences(Node* head, int value)
{
    Node* temp = head;
    int position = 1;
    int count = 0;

    cout << "Occurrences of " << value << " found at positions: ";

    while (temp != NULL)
    {
        if (temp->data == value)
        {
```

```
    cout << position << " ";  
    count++;  
}
```

```
temp = temp->next;  
position++;  
}
```

```
if (count == 0)  
{  
    cout << "No occurrence found.";  
}
```

```
cout << endl;
```

```
cout << "Total occurrences = " << count << endl;  
}
```

```
int main()  
{  
    Node* head = NULL;
```

```
// Create Linked List
```

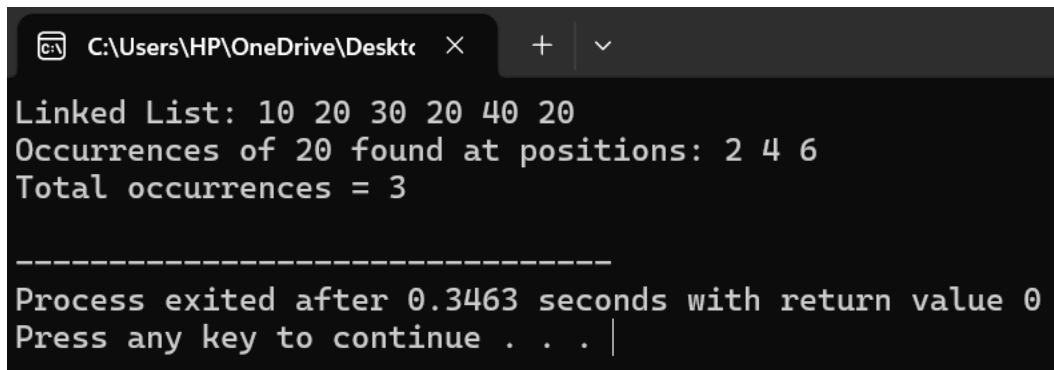
```
insertEnd(head, 10);  
insertEnd(head, 20);  
insertEnd(head, 30);  
insertEnd(head, 20);  
insertEnd(head, 40);  
insertEnd(head, 20);
```

```
// Display list
cout << "Linked List: ";
display(head);

// Find occurrences of 20
findOccurrences(head, 20);

return 0;
}
```

OUTPUT:



```
C:\Users\HP\OneDrive\Desktop >
Linked List: 10 20 30 20 40 20
Occurrences of 20 found at positions: 2 4 6
Total occurrences = 3

-----
Process exited after 0.3463 seconds with return value 0
Press any key to continue . . . |
```